

# AssetGuard

Military Asset Management System

## Project Documentation

Architecture, data model, role-based access control, API logging, setup, endpoints and platform access.

|                     |                                  |
|---------------------|----------------------------------|
| Project             | AssetGuard                       |
| Application Type    | Full-stack web application       |
| Backend             | Django + Django REST Framework   |
| Frontend            | React + Vite                     |
| Production Database | PostgreSQL                       |
| Authentication      | JWT (SimpleJWT)                  |
| Deployment          | Render (API) + Vercel (frontend) |

Prepared for project submission - October 2026

# 1. Project Overview

AssetGuard is a web-based Military Asset Management System designed to provide controlled, auditable tracking of critical assets across multiple bases. The application tracks inventory balances and operational movements such as purchases, transfers, assignments and expenditures. A dashboard summarizes opening balance, closing balance, net movement, assigned assets and expended assets, with filtering by base, equipment type and date.

## Core objectives

- Maintain visibility of available inventory by military base and equipment type.
- Record purchases and automatically increase inventory.
- Record inter-base transfers with both source and destination history.
- Record assignments and expenditures while reducing available inventory.
- Provide role-based access for administrators, base commanders and logistics officers.
- Maintain audit records for important asset transactions.

## Assumptions

- Each non-admin operational user is associated with a base.
- Inventory is maintained per unique base and equipment-type combination.
- Quantities are non-negative integer units.
- Authenticated users access APIs using JWT access tokens; refresh tokens are used to renew sessions.
- Production data is stored in PostgreSQL; the local development environment can run with SQLite.

## Current limitations

- The project is an initial framework and does not include advanced procurement approval workflows, barcode/RFID integration or real-time external logistics feeds.
- Historical opening/closing balances for arbitrary past dates are limited by the current transaction/inventory design; a snapshot or ledger-based model would provide stronger historical reconstruction.
- Deployment uses a free hosting tier, so the backend can take additional time to respond after a period of inactivity.
- The application is intended as a demonstration/assessment system, not as a production defense system handling classified information.

# 2. Tech Stack & Architecture

| Layer          | Technology                              | Reason for choice                                                                               |
|----------------|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| Frontend       | React + Vite                            | Component-based UI, fast development/build process and straightforward REST API integration.    |
| API / Backend  | Python, Django, Django REST Framework   | Mature framework, ORM, serializers, permissions and rapid REST API development.                 |
| Authentication | SimpleJWT                               | Stateless access/refresh token authentication suitable for a separate React frontend.           |
| Database       | PostgreSQL (production); SQLite (local) | PostgreSQL provides a robust relational production store; SQLite keeps local setup lightweight. |

| Layer       | Technology      | Reason for choice                                                                                 |
|-------------|-----------------|---------------------------------------------------------------------------------------------------|
| HTTP Client | Axios           | Centralized API base URL plus request/response interceptors for JWT attachment and token refresh. |
| Hosting     | Render + Vercel | Backend/API deployment on Render and React/Vite frontend deployment on Vercel.                    |

## High-level architecture

**Browser / React UI** → Axios REST calls with JWT → **Django REST API** → Django ORM → **PostgreSQL**. The frontend reads its API base URL from `VITE_API_BASE_URL`. The backend validates JWTs, applies role/base permissions, performs transaction logic and returns JSON responses. Production CORS is configured for the deployed frontend origin.

## 3. Data Models / Schema

| Entity        | Important fields                                                            | Relationship / purpose                                            |
|---------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------|
| Base          | name, location                                                              | Military base; referenced by users, inventory and transactions.   |
| User          | username, email, role, base, is_active                                      | Custom Django user; operational roles are associated with a base. |
| EquipmentType | name, category                                                              | Tracked equipment such as vehicles, weapons and ammunition.       |
| Inventory     | base, equipment_type, quantity                                              | Current stock; unique per Base + EquipmentType.                   |
| Purchase      | base, equipment_type, quantity, purchase_date, created_by                   | Inbound procurement; increases inventory.                         |
| Transfer      | from_base, to_base, equipment_type, quantity, transfer_date, created_by     | Moves stock between bases.                                        |
| Assignment    | base, equipment_type, personnel_name, quantity, assignment_date, created_by | Records issued assets and reduces available inventory.            |
| Expenditure   | base, equipment_type, quantity, reason, expenditure_date, created_by        | Records consumed assets and reduces inventory.                    |
| AuditLog      | user, action, base, equipment_type, quantity, details, timestamp            | Application transaction audit trail.                              |

**Key relationships:** Base 1-to-many User; Base and EquipmentType each 1-to-many Inventory; Base/EquipmentType/User are referenced by transaction records. Inventory enforces a unique Base + EquipmentType combination. Transfer has separate source and destination Base foreign keys.

## 4. RBAC Explanation

| Role           | Access level                                                                                                                                 |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Admin          | Full access across bases, including dashboard, assets, purchases, transfers, assignments, expenditures, audit logs and user management.      |
| Base Commander | Operational access restricted to the assigned base; can view relevant dashboard/assets and work with permitted assignment/expenditure flows. |

| Role              | Access level                                                                                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Logistics Officer | Logistics-focused access restricted to the assigned base; intended for purchases, transfers and related inventory/audit views; no user management. |

**Enforcement method:** Django REST Framework permission classes authenticate users and check roles. Querysets and transaction operations apply base restrictions for non-admin users. User Management remains Admin-only. React protected routes and role-aware navigation improve the UI experience, but backend permission checks are the security boundary.

## 5. API Logging / Transaction Logging

AssetGuard uses an **AuditLog** model to retain application-level transaction history. Audit entries can associate the acting user, action type, base, equipment type, quantity, descriptive details and timestamp. Supported transaction action categories include PURCHASE, TRANSFER, ASSIGNMENT and EXPENDITURE.

- Purchase logging records the actor and purchased asset movement.
- Transfer logging preserves movement context between bases.
- Assignment logging records the user responsible for issuing assets.
- Expenditure logging records consumed assets and operational context.
- The Audit Log API exposes records according to authenticated role/base permissions.

The deployed web service also produces HTTP request logs containing request paths, methods and response status codes. These are useful for operational troubleshooting, while AuditLog is the domain-level transaction history.

## 6. Setup Instructions

### Prerequisites

Python, pip/virtual environment, Node.js/npm and Git. PostgreSQL is required for a production-style database; local development can use SQLite.

### Backend

```
git clone <repository-url>
cd AssetGuard
python -m venv myenv
myenv\Scripts\Activate.ps1 # Windows PowerShell
pip install -r requirements.txt
cd backend
python manage.py migrate
python manage.py runserver
```

The local API runs at `http://127.0.0.1:8000/`. When `DATABASE_URL` is configured, the application can connect to PostgreSQL; otherwise the current local development configuration can use SQLite.

### Frontend

```
cd frontend
npm install
npm run dev
```

For local development, the Axios client falls back to `http://127.0.0.1:8000/api/`. For deployment, set `VITE_API_BASE_URL` to the production API base URL ending in `/api/`, then rebuild/redeploy the frontend.

### Production configuration

Production uses environment variables rather than committed secrets. Typical variables include SECRET\_KEY, DEBUG, DATABASE\_URL and CORS\_ALLOWED\_ORIGINS on the backend, and VITE\_API\_BASE\_URL on the frontend. Run migrations and static-file collection during backend deployment.

## 7. Key API Endpoints

| Method             | Endpoint              | Purpose                                                                  |
|--------------------|-----------------------|--------------------------------------------------------------------------|
| POST               | /api/auth/login/      | Authenticate and return JWT access/refresh tokens plus user information. |
| POST               | /api/auth/refresh/    | Renew access token.                                                      |
| GET                | /api/dashboard/       | Dashboard balances and movement metrics.                                 |
| GET                | /api/bases/           | List permitted bases.                                                    |
| GET                | /api/equipment-types/ | List equipment types.                                                    |
| GET                | /api/inventory/       | List current inventory; supports base filtering.                         |
| GET / POST         | /api/purchases/       | List/create purchases.                                                   |
| GET / POST         | /api/transfers/       | List/create transfers.                                                   |
| GET / POST         | /api/assignments/     | List/create assignments.                                                 |
| GET / POST         | /api/expenditures/    | List/create expenditures.                                                |
| GET                | /api/audit-logs/      | Retrieve audit logs; supports action filtering.                          |
| GET / POST         | /api/users/           | Admin list/create users.                                                 |
| GET / PUT / DELETE | /api/users//          | Admin retrieve/update/delete a user.                                     |

### Authentication header

Authorization: Bearer <access-token>

## 8. Login Credentials

The following administrator account was used for the deployed platform during final testing and provides Admin access to AssetGuard.

| Field    | Value     |
|----------|-----------|
| Username | admin     |
| Password | Admin@123 |
| Role     | ADMIN     |

**Security note:** These credentials are included because the submission explicitly requests a working login. They should be changed or removed after evaluation. Production secrets such as SECRET\_KEY, DATABASE\_URL and JWT tokens must not be included in the submitted source archive or documentation.

## Deployment Summary

The submitted version uses a React/Vite frontend deployed on Vercel and a Django REST API deployed on Render, backed by PostgreSQL. The system was verified with authenticated API requests and migrated project data, including bases, users, equipment types, inventory and transaction records.

End of document